

client container

Docker host, one Compose network

CLI / load generator
`bench.loadgen`

WordCountClient
RPyC client stub

*RPyC over TCP, synchronous request-reply,
one TCP connection per request:
request `count(text, keyword)`,
reply `(count, server, cache hit)`*

server1 container (1 CPU)

RPyC ThreadedServer (one thread per connection)

WordCountService: validate request →
cache lookup → on a miss: count and store

TextCatalog
texts tokenized at startup

CountCache
cache-aside Redis adapter

*file read once
at startup*

*RESP over TCP: GET + ZINCRBY
pipelined, SET on a miss*

corpus volume (read-only)
8 books, one .txt per text ID

redis container (in-memory)

counts `wc:count:text:word`

hot keywords `wc:hot` (sorted set)